Spring MVC Security - JDBC Authentication - BCrypt Encryption - Overview & Coding

This lesson is about integrating Spring Security with password encryption. As of now, we've been storing passwords in plain text. This might be acceptable for initial learning phases, but it's certainly not suitable for production or real-world applications. The industry best practice is to store passwords in encrypted form.

Consider our example users: John, Marion, and Susan. What we'll see next are their encrypted passwords. If our database was compromised, attackers wouldn't discern the original passwords, because they are stored in encrypted format.

username	password	enabled
john	{bcrypt}$2a$10$qeS0HEh7urweMojsnwNAR.vcXJeXR1UcMRZ2WcGQl9YeuspUdgF.q	1
mary	{bcrypt}$2a$10$qeS0HEh7urweMojsnwNAR.vcXJeXR1UcMRZ2WcGQl9YeuspUdgF.q	1
susan	{bcrypt}$2a$10$qeS0HEh7urweMojsnwNAR.vcXJeXR1UcMRZ2WcGQl9YeuspUdgF.q	1

The Spring Security team endorses the bcrypt algorithm. Why? Bcrypt offers a one-way encryption hash. It incorporates a random salt with the password for extra security and contains mechanisms to thwart brute force attacks. Thus, it's not only recommended by the Spring Team but is also widely employed across numerous projects.

For those interested in delving deeper into bcrypt:

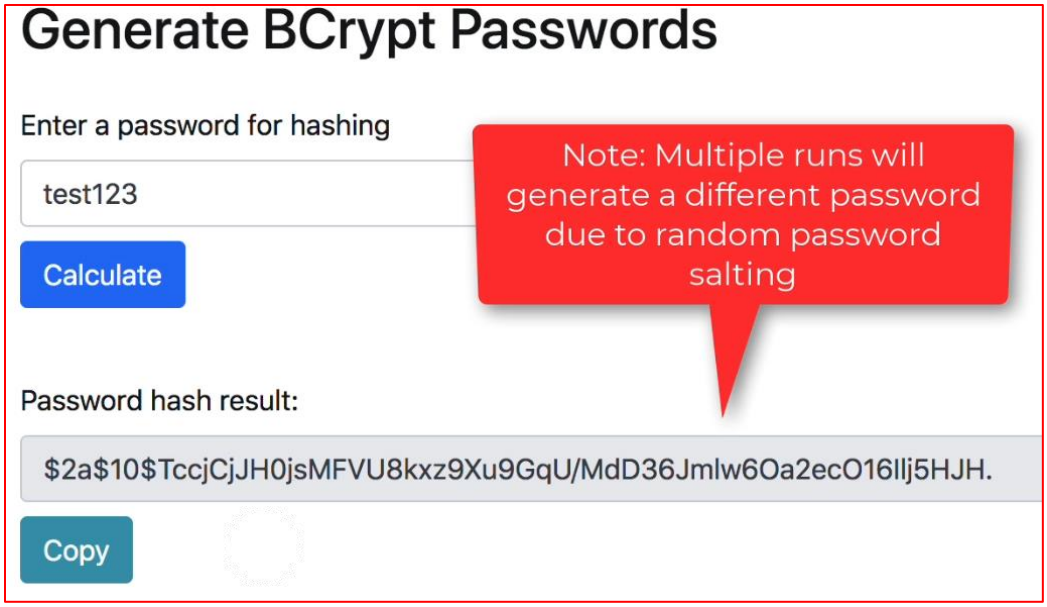
- Why use bcrypt for password hashing? Visit why-bcrypt
- For a comprehensive analysis of the bcrypt algorithm, go to bcrypt-wiki-page
- For best practices on password hashing, check password-hashing-best-practices

Now, how do you encrypt a plain text password using bcrypt? Two primary methods exist:

1. Utilize an online encryption tool.
2. Implement encryption programmatically using Java.

In this lesson, we're focusing on the first approach. For those intrigued by the latter, stay tuned for upcoming lessons.

For a live bcrypt encryption demo, navigate to generate-bcrypt-password. Here, input your plain text password, and the tool will provide the encrypted version. Let's run a quick simulation. I'll enter `'test123'` and, upon clicking `'calculate'`, we get its bcrypt hash.

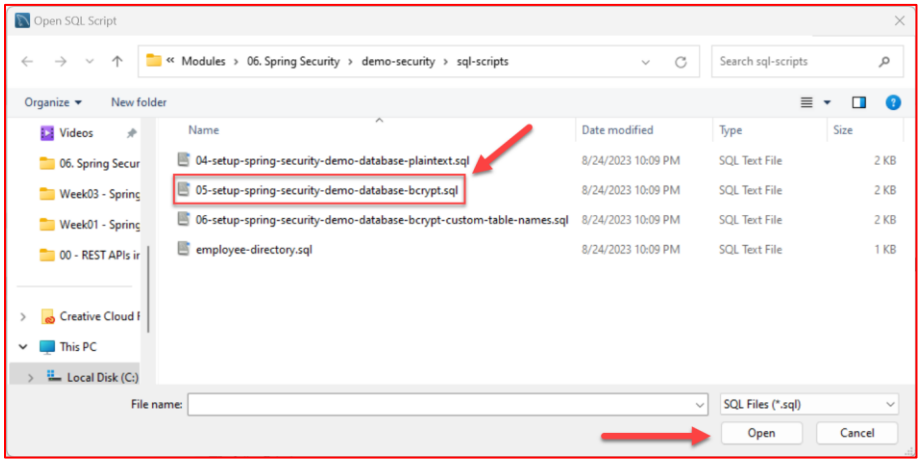


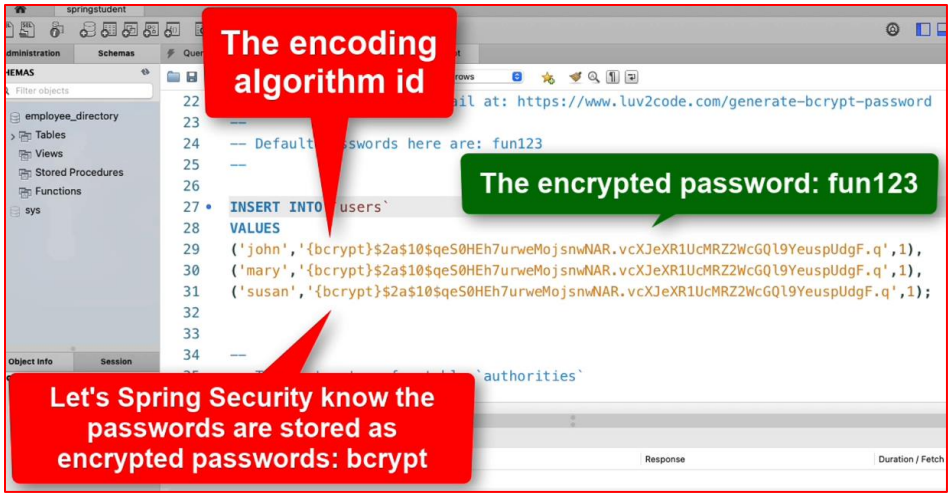
Note: Generating the hash multiple times for the same password will produce varied results due to the randomness of salting. Salt, in cryptography, refers to random data that's combined with passwords to ensure uniqueness.

With encrypted passwords, you can bolster your user account security.

Development process

Let's dive into our development process. First things first, we need to execute the SQL script that houses the encrypted passwords. Next, we must adjust the DDL for the password field to ensure it's set to a length of 68 characters. Why? That's because the password's encryption will demand this space. And the good news? No changes are required in the Java source code. It's predominantly a configuration adjustment.





The term `'bcrypt'` will come up frequently. This corresponds to the ID we've assigned for passwords in our database table. The encoded password will represent the hashed value, either sourced from a website or generated via Java code.

A critical note on the password column: it must be set to at least 68 characters. Here's the breakdown - 8 characters for the `'bcrypt'` identifier, and the actual encrypted password is always 60 characters, irrespective of the plaintext input's length.

Now, let's alter the DDL for the password field. The `'password'` column will be set as CHAR(68). This is essential to accommodate the encrypted passwords. For example, consider user John with the plaintext password `'fun123'`. His encrypted password will be stored with the `'bcrypt'` prefix, signaling to Spring Security that bcrypt encryption has been applied. While in the past, we used plain text passwords, in this case, bcrypt is our chosen encryption method. The same process will be repeated for users Mary and Susan.

Let's take a closer look at Spring Security's login process:

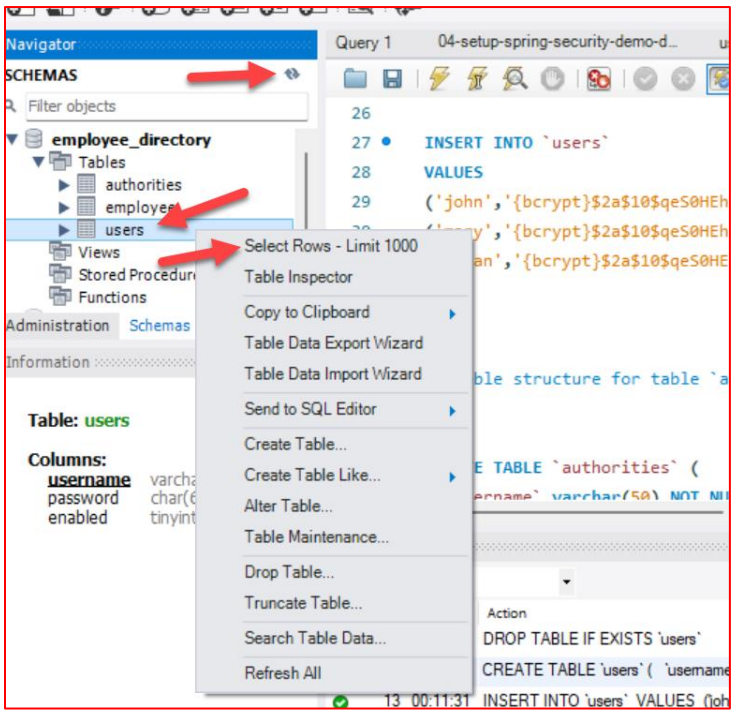
1. A user enters their plaintext password into the login form.
2. Spring Security Filters process the input using JDBC authentication.
3. The system retrieves the password from the database and recognizes the encoding algorithm, i.e., bcrypt.
4. It encrypts the entered plaintext password, leveraging the salt from the stored password.
5. It then compares the newly encrypted password with the stored encrypted password.
6. If there's a match, voila, login is successful; otherwise, access is denied.

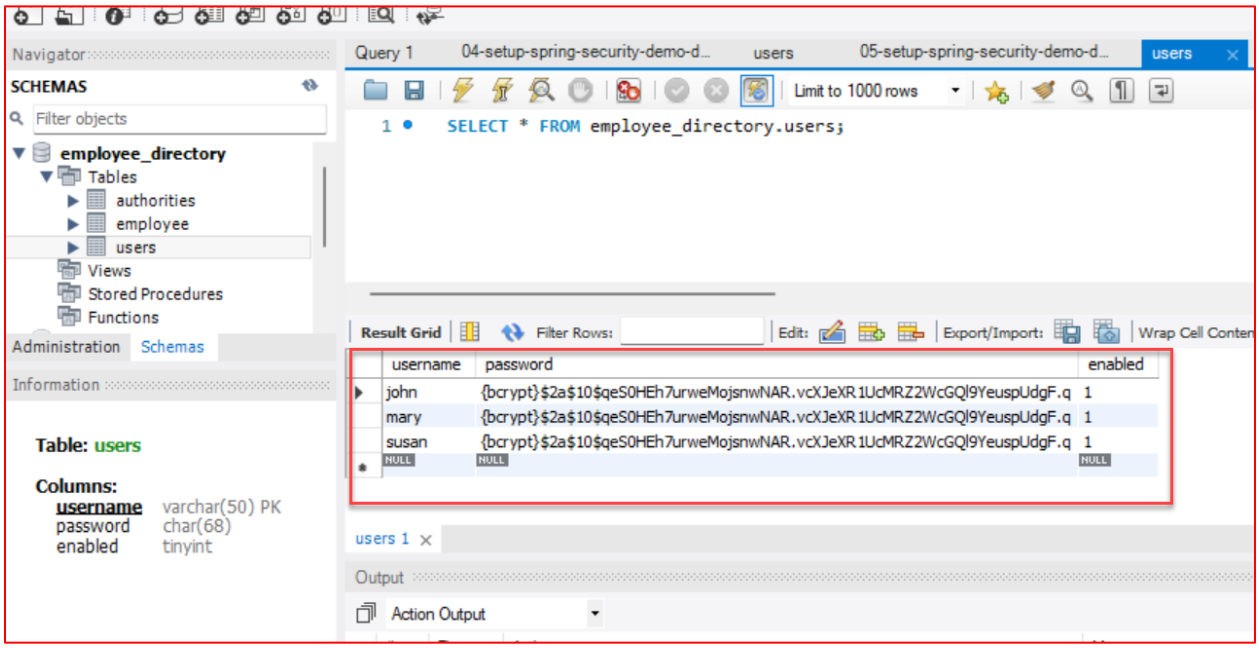
Remember, bcrypt is a one-way encryption algorithm. Therefore, it's impossible to decrypt and retrieve the original password from the database.

Now, let's transition to our next practical step. Navigate to our provided directory, `sql-scripts`, and open the SQL script within the MySQL Workbench. We'll execute the script, setting up our database tables and embedding encrypted passwords for our users.

Ensure to use the exact length for the password field, CHAR(68), to facilitate bcrypt encryption.

After executing the script, you can check the database to ensure that encrypted passwords are correctly stored for John, Mary, and Susan.





Let's then test the login mechanism within our application. Logging in with the wrong password will fail, of course. However, using the correct password, such as '`fun123`' for John, should grant access. With bcrypt encryption seamlessly integrated, it's a job well done!